Universidad del Valle de Guatemala
Programación Orientada a Objetos, Sección 30
Facultad de Ingeniería

Laboratorio 03 Herencia

Emmanuel Alejandro Montufar Pinto – 26805
Andrés Martínez Torres – 26633

Requisitos funcionales

Información que debe conservar el sistema

El sistema debe conservar los envíos registrados. Cada envío tendrá un código único, remitente, destinatario, peso real, tipo de destino, estado actual y número de intentos realizados.

Los paquetes también conservarán largo, ancho y alto. La condición de paquete frágil se representará mediante el atributo booleano fragil. Cuando este valor sea verdadero, también se conservará el valor declarado utilizado para calcular el seguro. Las cargas refrigeradas conservarán la temperatura requerida.

De cada vehículo se conservarán la placa única, la capacidad máxima en kilogramos, la indicación de refrigeración y su disponibilidad. De cada despacho se conservarán el vehículo, los envíos asignados, el estado y los resultados registrados. Asimismo, el sistema debe conservar los ingresos por registro y por reintentos.

Operaciones que debe permitir

- Cotizar un envío sin registrarlo ni alterar los ingresos.
- Registrar un envío aceptado y cobrar el costo completo.
- Registrar y consultar vehículos.
- Crear despachos, asignar envíos y retirarlos mientras el despacho esté en preparación.
- Hacer salir un despacho y registrar el resultado de cada entrega.
- Consultar envíos y generar los reportes solicitados.

Condiciones de validez

Los códigos y placas no pueden estar vacíos ni repetidos. Los pesos, dimensiones, capacidades y valores declarados que correspondan deben ser positivos. Un sobre no puede superar 0.5 kg. Una carga refrigerada debe tener una temperatura entre -25 °C y 8 °C y solamente puede tener destino urbano.

Para crear un despacho, el vehículo debe estar disponible. Para asignar un envío, este debe encontrarse en bodega, no estar asignado a otro despacho, ser compatible con el vehículo y no provocar que la suma de pesos reales supere la capacidad máxima. El peso volumétrico no se utiliza para la capacidad del vehículo.

Solo se pueden retirar envíos mientras el despacho se encuentra en preparación. Un despacho no puede salir vacío. Los resultados únicamente pueden registrarse cuando el despacho está en ruta y cada envío puede recibir un solo resultado dentro del despacho. Toda operación inválida se rechaza sin modificar el estado previo ni los ingresos.

Decisiones que dependen de la categoría

Costo del envío. Sobre, Paquete y CargaRefrigerada implementan su propio método calcularCosto. Sobre utiliza una tarifa fija según el destino. Paquete calcula la tarifa base más el peso facturable y, cuando fragil es verdadero, agrega Q15 de embalaje y el 3 % del valor declarado. CargaRefrigerada utiliza la tarifa urbana, Q12 por kilogramo real y Q60 de cadena de frío.

Tiempo estimado. Cada subclase implementa calcularDiasEntrega. Sobre devuelve uno o dos días según el destino. Paquete devuelve dos o cuatro días y agrega un día cuando fragil es verdadero. CargaRefrigerada siempre devuelve un día.

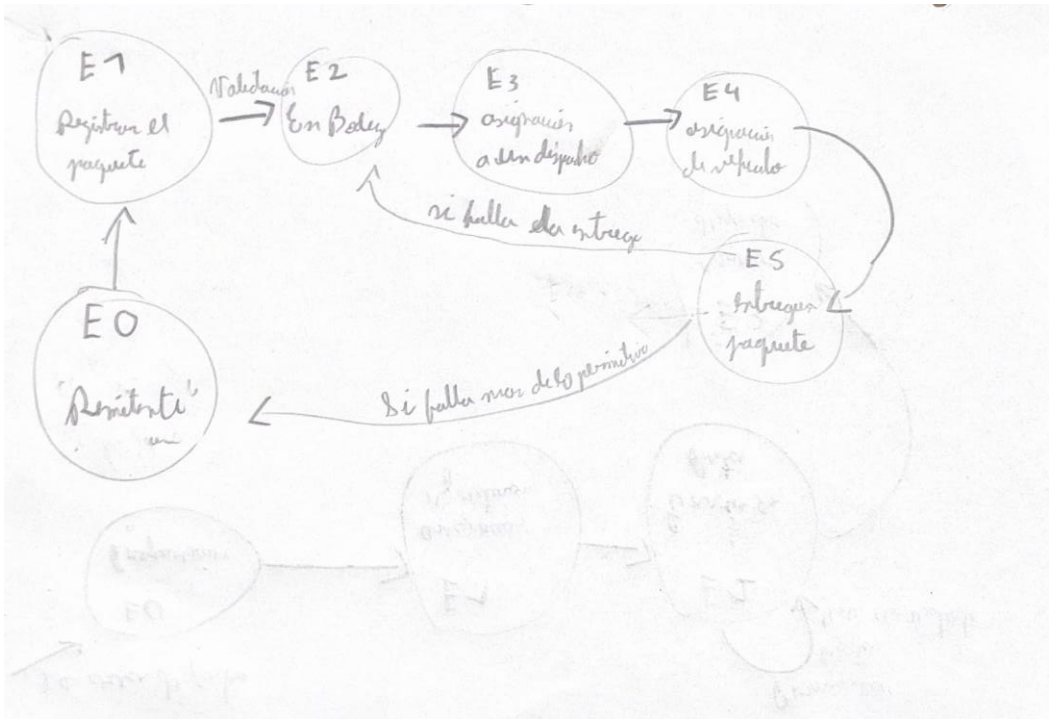
Política de intentos. Cada subclase implementa getMaximoIntentos. Sobre devuelve tres. Paquete devuelve tres cuando fragil es falso y dos cuando es verdadero. CargaRefrigerada devuelve uno.

Compatibilidad con vehículos. Envio define esCompatibleCon y permite cualquier vehículo por defecto. CargaRefrigerada sobrescribe el método para exigir refrigeración. Despacho consulta este método sin preguntar el tipo concreto del objeto.

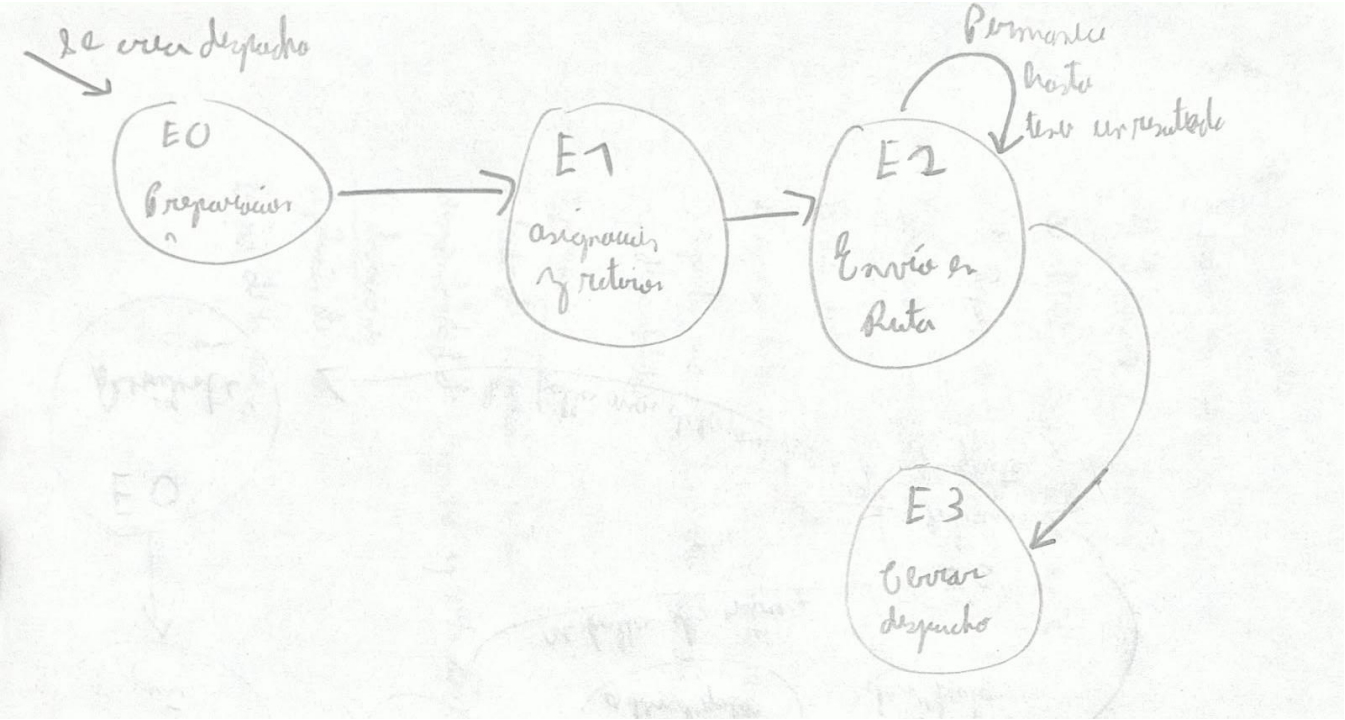
Categoría para reportes. Cada subclase implementa getCategoria. En Paquete, el resultado será Paquete o Paquete frágil según el flag. Esto permite agrupar los envíos sin utilizar instanceof.

Estados y transiciones

Estados del envío



Estados del despacho



Clases propuestas

Envio será una clase abstracta que conservará los datos comunes, el estado y los intentos. También declarará los métodos que cambian según la categoría.

Sobre heredará de Envio y validará el peso máximo. Además, implementará sus reglas de costo, tiempo e intentos.

Paquete heredará de Envio y conservará las dimensiones, el flag fragil y el valor declarado. Calculará el peso volumétrico y aplicará las reglas del paquete normal o frágil según el flag.

CargaRefrigerada heredará de Envio. Validará la temperatura y el destino, calculará su tarifa y exigirá un vehículo refrigerado.

Vehiculo será una clase concreta con placa, capacidad y refrigeración. No se propone una clase abstracta de flotilla porque el problema actual no define comportamientos distintos entre tipos de vehículo.

Despacho administrará el vehículo y la colección de envíos. Validará la capacidad, coordinará la salida, registrará los resultados y liberará el vehículo al cerrar.

SistemaEnvios conservará las colecciones, verificará códigos y placas únicas, registrará los ingresos y generará los reportes. Aplicacion contendrá main, el menú y la lectura segura de entradas.

Atributos y métodos

Las siguientes son las únicas tablas del análisis. La visibilidad se representa con - para privado, # para protegido y + para público.

Clase	Visibilidad	Atributo o método	Tipo, parámetros y propósito
Envio	-	codigo: String	Identificador del envío.
Envio	-	remitente: String	Nombre del remitente.
Envio	-	destinatario: String	Nombre del destinatario.
Envio	-	pesoRealKg: double	Peso utilizado para la capacidad.
Envio	-	destino: TipoDestino	URBANO o DEPARTAMENTAL.
Envio	-	estado: EstadoEnvio	Estado actual del envío.
Envio	-	intentosRealizados: int	Cantidad de intentos iniciados.
Envio	#	Envio(datos comunes)	Valida e inicializa el envío en bodega.
Envio	+	calcularCosto(): double	Método abstracto para el costo de registro.
Envio	+	calcularDiasEntrega(): int	Método abstracto para el tiempo estimado.
Envio	+	getMaximoIntentos(): int	Método abstracto para la política de intentos.
Envio	+	getCategoria(): String	Método abstracto utilizado por reportes.
Envio	+	esCompatibleCon(Vehiculo): boolean	Permite cualquier vehículo por defecto.
Envio	+	asignar(), retirar(), iniciarIntento()	Controlan las transiciones previas a la entrega.
Envio	+	registrarEntrega(boolean): void	Pasa a entregado, bodega o devuelto.

Clase	Visibilidad	Atributo o método	Tipo, parámetros y propósito
Sobre	+	Sobre(datos comunes)	Llama a super y valida peso máximo de 0.5 kg.
Sobre	+	calcularCosto(): double	Q20 urbano o Q35 departamental.
Sobre	+	calcularDiasEntrega(): int	Uno o dos días según destino.
Sobre	+	getMaximoIntentos(): int	Devuelve tres.
Paquete	-	largoCm, anchoCm, altoCm: double	Dimensiones positivas del paquete.
Paquete	-	fragil: boolean	Distingue paquete normal y paquete frágil.
Paquete	-	valorDeclarado: double	Se exige cuando fragil es verdadero.
Paquete	+	Paquete(datos comunes, dimensiones, fragil, valor)	Llama a super y valida sus datos.
Paquete	+	calcularPesoVolumetrico(): double	Calcula largo por ancho por alto dividido 5000.
Paquete	+	calcularKgFacturables(): int	Redondea hacia arriba el mayor peso.
Paquete	+	calcularCosto(): double	Aplica base y peso; si es frágil agrega embalaje y seguro.
Paquete	+	calcularDiasEntrega(): int	Devuelve dos o cuatro días y suma uno si es frágil.
Paquete	+	getMaximoIntentos(): int	Devuelve dos si es frágil y tres si no lo es.
Paquete	+	getCategoria(): String	Devuelve Paquete o Paquete frágil.
CargaRefrigerada	-	temperaturaRequerida: double	Temperatura entre -25 °C y 8 °C.
CargaRefrigerada	+	CargaRefrigerada(datos, temperatura)	Llama a super y exige destino urbano.
CargaRefrigerada	+	calcularCosto(): double	Q25 más Q12 por kg real más Q60.
CargaRefrigerada	+	calcularDiasEntrega(): int	Devuelve un día.
CargaRefrigerada	+	getMaximoIntentos(): int	Devuelve un intento.
CargaRefrigerada	+	esCompatibleCon(Vehiculo): boolean	Exige refrigeración.

Clase	Visibilidad	Atributo o método	Tipo, parámetros y propósito
Vehiculo	-	placa: String	Identificador único del vehículo.
Vehiculo	-	capacidadMaximaKg: double	Capacidad máxima positiva.
Vehiculo	-	refrigerado: boolean	Indica si posee refrigeración.
Vehiculo	-	disponible: boolean	Indica si puede recibir un despacho.
Vehiculo	+	Vehiculo(placa, capacidad, refrigerado)	Inicializa el vehículo disponible.
Vehiculo	+	ocupar(), liberar(): void	Modifican la disponibilidad.
Despacho	-	vehiculo: Vehiculo	Vehículo reservado.
Despacho	-	envios: List<Envio>	Envíos asignados.
Despacho	-	estado: EstadoDespacho	Estado actual del despacho.
Despacho	-	resultados: Set<String>	Evita registrar resultados duplicados.
Despacho	+	asignarEnvio(Envio): void	Valida estado, capacidad y compatibilidad.
Despacho	+	retirarEnvio(Envio): void	Retira un envío durante la preparación.
Despacho	+	getPesoCargado(): double	Suma los pesos reales.
Despacho	+	getCapacidadDisponible(): double	Calcula la capacidad restante.
Despacho	+	salir(): void	Inicia intentos y cambia a ruta.
Despacho	+	registrarResultado(Envio, boolean): void	Aplica el resultado y verifica el cierre.
SistemaEnvios	-	envios, vehiculos, despachos	Colecciones principales del sistema.
SistemaEnvios	-	ingresoRegistro, ingresoReintentos	Ingresos separados por origen.
SistemaEnvios	+	cotizar(...): Cotizacion	Calcula sin conservar ni cobrar.
SistemaEnvios	+	registrarEnvio(Envio): void	Verifica el código y registra el ingreso.
SistemaEnvios	+	registrarVehiculo(Vehiculo): void	Verifica la placa y agrega el vehículo.
SistemaEnvios	+	crearDespacho(Vehiculo): Despacho	Exige que el vehículo esté disponible.
SistemaEnvios	+	generarReportes(): void	Genera los tres reportes solicitados.

Colaboración de constructores

Los constructores de Sobre, Paquete y CargaRefrigerada llaman a super para inicializar los datos comunes definidos por Envio. Después, cada constructor valida sus propios datos. En Paquete, si fragil es falso, el valor declarado no participa en el costo; si fragil es verdadero, debe ser positivo.

Justificación del flag para paquete frágil

En este diseño no existe una clase PaqueteFragil. Paquete contiene el atributo booleano fragil porque el paquete frágil conserva los mismos datos y cálculos fundamentales del paquete normal. El flag activa el cargo de embalaje y seguro, un día adicional de entrega y un máximo de dos intentos.

Esta alternativa reduce la cantidad de clases y mantiene juntas las reglas de dos variantes cercanas. Como desventaja, calcularCosto, calcularDiasEntrega y getMaximoIntentos incluyen una condición interna. Si el paquete frágil adquiere más atributos o comportamientos propios, sería conveniente reemplazar el flag por una subclase PaqueteFragil que herede de Paquete.

No se necesita un segundo flag para paquete normal: fragil igual a falso representa un paquete normal y fragil igual a verdadero representa un paquete frágil.

Incorporación de una nueva categoría

Para agregar una categoría con reglas propias, como EnvioMascota, se crea una nueva subclase de Envio con sus atributos y validaciones. La nueva clase implementa calcularCosto, calcularDiasEntrega, getMaximoIntentos, getCategoria y esCompatibleCon cuando posea restricciones vehiculares.

Aplicacion debe incorporar la opción para solicitar los datos de la nueva categoría. Despacho no necesita cambiar porque trabaja con referencias de tipo Envio y llama a sus métodos polimórficos. Los reportes tampoco necesitan cambiar si agrupan mediante getCategoria y getEstado. Vehiculo solo se modificaría si la nueva categoría exige una característica que actualmente no está representada.